



Universiteit
Leiden

Machine Learning

The Art and Science of Algorithms that Make Sense of Data

Dr. Mitra Baratchi

27 February 2026

Leiden Institute of Advanced Computer Science - Leiden University

Recap of session 1-3

- Different types of machine learning tasks: supervised, unsupervised, predictive, and descriptive
- Evaluating the performance of different machine learning tasks
 - Contingency tables
 - Evaluation metrics
 - ROC curves
- Overfitting, bias-variance dilemma

Session 4: Tree models ¹

Reading material

→ Chapter 5, Machine learning: the art and science of algorithms that make sense of data

¹The slides partially include material based on the slides accompanying the book: ***Machine learning: the art and science of algorithms that make sense of data*** Cambridge University Press, 2012.

Table of Contents

1. Tree models
2. Decision trees
 - Growing a decision tree
 - Pruning
 - Sensitivity to skewed class distributions
3. Regression trees

Tree models

Tree models²

Tree models are among the most popular models in supervised learning. These are three types of models based on trees.

- Decision trees
- Random forest
- Gradient boosting

Decision Tree



Random Forest



Gradient Boosted Trees



Boosted Forest



Models based on trees

²Image source: <https://hpccsystems.com/blog/learning-trees-guide-to-decision-tree-based-machine-learning>

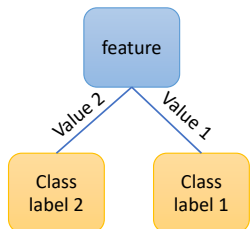
Tree models

- Properties
 - Understandable
 - Interpretable, logical
 - Non-linear

Decision trees

A decision tree

A decision tree is a tree-like graph with nodes representing the place where we pick a feature and ask a question; edges represent the answers to the question, and the leaves represent the actual output or the class label.



Essential elements of a tree

A decision tree

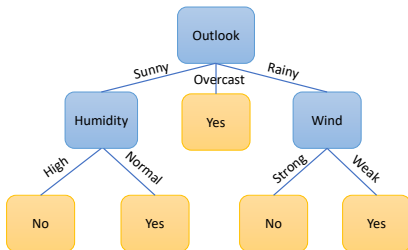
Let's look at an example ...

Play tennis?

Instance	Outlook	Temperature	Humidity	Wind	Play
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rainy	Mild	High	Weak	Yes
5	Rainy	Cool	Normal	Strong	Yes
6	Rainy	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rainy	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rainy	Mild	High	Strong	No

Weather and Play Tennis Dataset.

An example decision tree

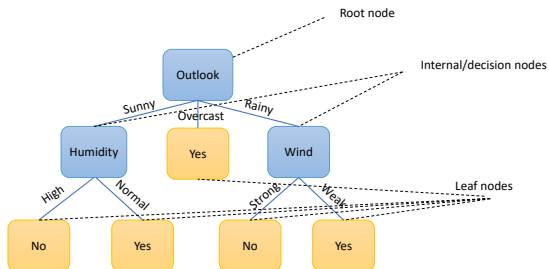


A decision tree made for the Weather and Play Tennis Dataset

All internal nodes are features in the table.

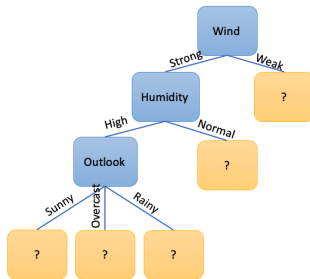
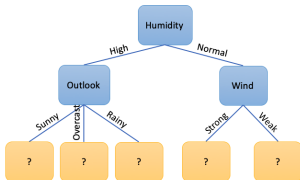
→ Some of the features in the table are not included

An example decision tree



A decision tree made for the Weather and Play Tennis Dataset

Other decision trees



(left) The first decision tree from the weather and play tennis dataset **(right)** The second decision tree from the same dataset

Max depth and number of leaves

What are the max **depth** and max **number of leaves** of a decision tree with d number of binary features?

Max depth and number of leaves

What are the max **depth** and max **number of leaves** of a decision tree with d number of binary features?

- The root node branches $\rightarrow (x_1 = 0 \text{ or } 1)$
- The next layer branches $\rightarrow (x_2 = 0 \text{ or } 1)$
- Branches of nodes at depth $i \rightarrow (x_{i+1} = 0 \text{ or } 1)$

Max depth and number of leaves

What are the max **depth** and max **number of leaves** of a decision tree with d number of binary features?

- The root node branches $\rightarrow (x_1 = 0 \text{ or } 1)$
- The next layer branches $\rightarrow (x_2 = 0 \text{ or } 1)$
- Branches of nodes at depth $i \rightarrow (x_{i+1} = 0 \text{ or } 1)$

There is a path from the root to each leaf $(x_1 = 0 \text{ or } 1), (x_2 = 0 \text{ or } 1), \dots, (x_d = 0 \text{ or } 1)$. We can allocate one leaf to any possible combination of values for $x_1, x_2, \dots, x_d \rightarrow 2^d$ leaves

Max depth and number of leaves

What are the max **depth** and max **number of leaves** of a decision tree with d number of binary features?

- The root node branches $\rightarrow (x_1 = 0 \text{ or } 1)$
- The next layer branches $\rightarrow (x_2 = 0 \text{ or } 1)$
- Branches of nodes at depth $i \rightarrow (x_{i+1} = 0 \text{ or } 1)$

There is a path from the root to each leaf $(x_1 = 0 \text{ or } 1), (x_2 = 0 \text{ or } 1), \dots, (x_d = 0 \text{ or } 1)$. We can allocate one leaf to any possible combination of values for $x_1, x_2, \dots, x_d \rightarrow 2^d$ leaves

Decision tree

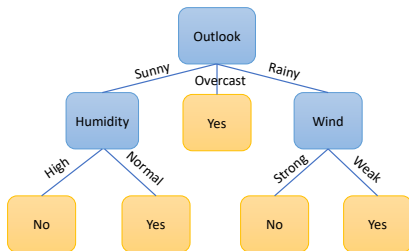
Any binary classifier with d binary features $h : \{0, 1\}^d \rightarrow \{0, 1\}$ can be represented with a decision tree with 2^d leaves and dept of $d + 1$.

Decision tree

Decision tree

A *decision tree* is a tree such that each internal node (the nodes that are not leaves) is labelled with a feature, and each edge emanating from an internal node is labelled with a literal. The set of literals at a node is called a *split*. Each leaf of the tree represents a *logical expression*, which is the conjunction of literals encountered on the path from the root of the tree to the leaf.

Logical expression



A decision tree

A logical expression of this decision tree:

$$\neg ((\text{Outlook} = \text{Sunny}) \wedge (\text{Humidity} = \text{High})) \vee$$
$$((\text{Outlook} = \text{Sunny}) \wedge (\text{Humidity} = \text{Normal})) \vee$$
$$(\text{Outlook} = \text{Overcast}) \vee$$
$$\neg ((\text{Outlook} = \text{Rainy}) \wedge (\text{Wind} = \text{Strong})) \vee$$
$$((\text{Outlook} = \text{Rainy}) \wedge (\text{Wind} = \text{Weak}))$$

Decision tree algorithms

How to learn a decision tree?

Table of content

1. Tree models
2. Decision trees
 - Growing a decision tree
 - Pruning
 - Sensitivity to skewed class distributions
3. Regression trees

Growing a decision tree

The general procedure:

- Start from the top of the tree, place the best feature as the *root* that *splits* the training set
- Select the next feature to *split* the training set
- *Repeat* the previous steps to find *leaf* nodes (set the majority vote of data in the leaf nodes as a class label)

Growing a feature tree

Algorithm $\text{GrowTree}(D, F) \leftarrow$.

In : data D ; set of features F .

O : feature tree T with labelled leaves.

```
1 if  $\text{Homogeneous}(D)$  then return  $\text{Label}(D)$ ;  
2  $S \leftarrow \text{BestSplit}(D, F)$  split  $D$  into subsets  $D_i$  according to the literals in  $S$ ;  
3 for each  $i$  do  
4   | if  $D_i \neq \emptyset$  then  $T_i \leftarrow \text{GrowTree}(D_i, F)$  ;  
5   | else  $T_i$  is a leaf labelled with  $\text{Label}(D)$ ;  
6 end  
7 return a tree whose root is labelled with  $S$  and whose children are  $T_i$ 
```

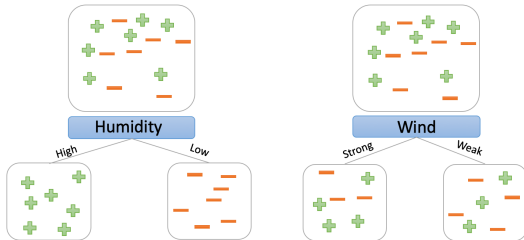
Please note that this is a *recursive* function.

Growing a decision tree

- How do we determine what is the best *split*?

Best split

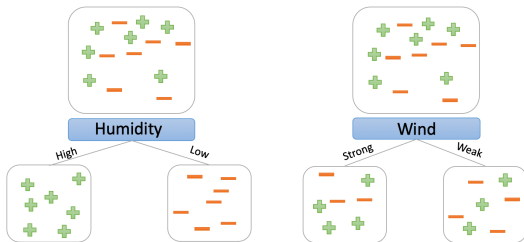
How to decide what is the best *split*?



(left) A pure split (right) an impure split

Best split

How to decide what is the best *split*?

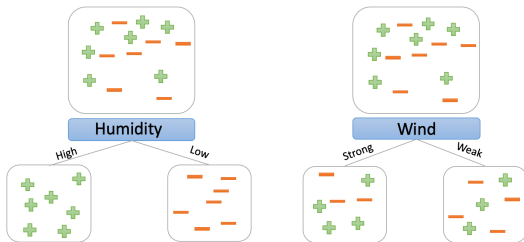


(left) A pure split (right) an impure split

- To assign the majority class label to leaves, we look for a clear split between two classes

Best split

How to decide what is the best *split*?

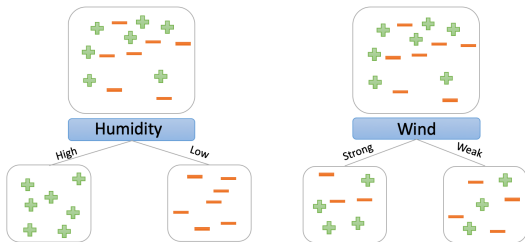


(left) A pure split (right) an impure split

- To assign the majority class label to leaves, we look for a clear split between two classes → *Purity* of the children of the split

Best split

How to decide what is the best *split*?



(left) A pure split (right) an impure split

- To assign the majority class label to leaves, we look for a clear split between two classes → *Purity* of the children of the split

→ The scenario on the right is more common. So we need to somehow compare features based on their impurities.

Measuring impurity

Impurity

Total *impurity* of a set of leafs D_j with $\text{Imp}(D_j)$, where $D = D_1 \cup \dots \cup D_l$ is

$$\text{Imp}(\{D_1, \dots, D_l\}) = \sum_{j=1}^l \frac{|D_j|}{|D|} \text{Imp}(D_j)$$

Measuring impurity

Impurity

Total *impurity* of a set of leafs D_j with $\text{Imp}(D_j)$, where $D = D_1 \cup \dots \cup D_l$ is

$$\text{Imp}(\{D_1, \dots, D_l\}) = \sum_{j=1}^l \frac{|D_j|}{|D|} \text{Imp}(D_j)$$

How to calculate $\text{Imp}(D_j)$?

Impurity measures

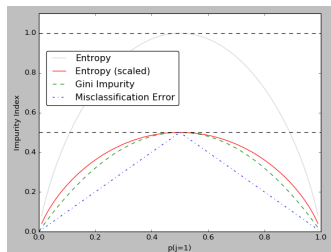
Assume the *empirical probability* of the positive classes:

$$\dot{p} = n^{\oplus} / (n^{\oplus} + n^{\ominus})$$

We can use this metric to calculate different *impurity* metrics:

- **Minority class (or misclassification error):** $\min(\dot{p}, 1 - \dot{p})$
- **Entropy:** $-\dot{p} \log_2 \dot{p} - (1 - \dot{p}) \log_2 (1 - \dot{p})$
- **Gini index:** $2\dot{p}(1 - \dot{p})$

Impurity measures



Comparison of different impurity measures³

Gini index and Entropy

Both the *entropy* and the *Gini index* are smooth and concave upper bounds of the training error. These properties can be advantageous in some situations.

³Image source: <https://www.bogotobogo.com/python/scikitlearn/>

Entropy

Let's now take a deeper look at entropy:

$$-\dot{p} \log_2 \dot{p} - (1 - \dot{p}) \log_2 (1 - \dot{p})$$

Entropy

Entropy of an event refers to the amount of information in terms of the number of bits needed to encode the probability of that event.

→ $I = -\log_2 p$ bits is needed to encode an event with probability p

- Higher probability → less number of bits (e.g., $p = 1 \rightarrow 0$ bit)
- Lower probability → more number of bits (e.g., $p = \frac{1}{2} \rightarrow 1$ bit)

What happens if we have more than one event (or possible outcomes)?

→ we can calculate their mean entropy.

Entropy

Entropy

Entropy is the expected average level of *information*, *surprise*, or *uncertainty* inherent in all the n possible outcomes of an event;

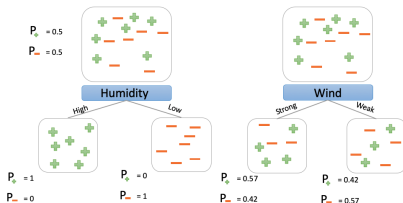
$$H(p_1, \dots, p_n) = \sum_{i=1}^n p(x_i) I(x_i) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$$

Entropy of binary classification

Entropy for a binary classification task where $\dot{p} = n^{\oplus} / (n^{\oplus} + n^{\ominus})$ is

$$H(\dot{p}, 1 - \dot{p}) = -\dot{p} \log_2 \dot{p} - (1 - \dot{p}) \log_2 (1 - \dot{p})$$

Comparing two splits

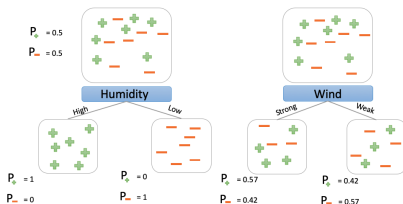


(left) Entropy of the split is 0, (right) Entropy of the split is 0.99

Total *impurity* of a set of leafs D_j with $\text{Imp}(D_j)$ is

$$\text{Imp}(\{D_1, \dots, D_l\}) = \sum_{j=1}^l \frac{|D_j|}{|D|} \text{Imp}(D_j)$$

Comparing two splits



(left) Entropy of the split is 0, (right) Entropy of the split is 0.99

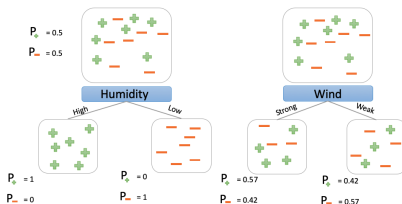
Total *impurity* of a set of leafs D_j with $\text{Imp}(D_j)$ is

$$\text{Imp}(\{D_1, \dots, D_l\}) = \sum_{j=1}^l \frac{|D_j|}{|D|} \text{Imp}(D_j)$$

→ The first split is better by having less entropy (more information) impurity and more purity.

Purity gain

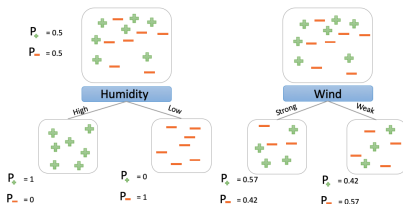
Other than comparing two splits, how do we assess if a split is useful at all?



Two different splits.

Purity gain

Other than comparing two splits, how do we assess if a split is useful at all?



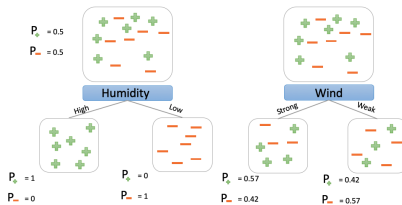
Two different splits.

Purity/information gain

In assessing the quality of a feature for splitting a parent node D into leaves $D_1, \dots, D_l$, it is customary to look at the *purity gain*.

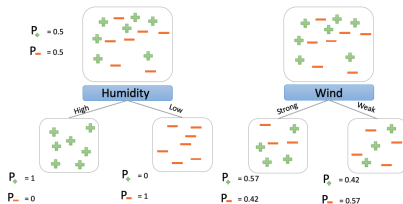
$$\text{Imp}(D) - \text{Imp}(\{D_1 \dots D_l\})$$

Purity gain



Two different splits.

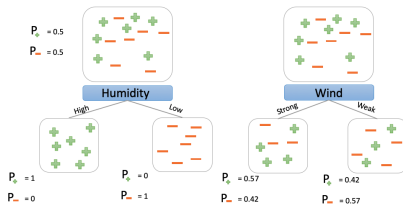
Purity gain



Two different splits.

Purity gain = original entropy - entropy after splitting

Purity gain

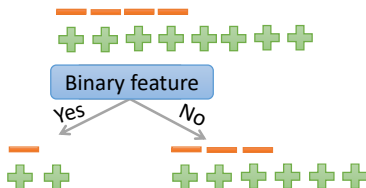


Two different splits.

Purity gain = original entropy - entropy after splitting

- Humidity $\rightarrow 1 - (\frac{7}{14}(0) + \frac{7}{14}(0)) = 1$
- Wind $\rightarrow 1 - (\frac{7}{14}(0.99) + \frac{7}{14}(0.99)) = 0.01$

Purity gain



A split with minimum purity gain.

Purity gain

Minimum purity gain is achieved when each child has the same output label distribution as the parent.

Continuous features

What about continuous features? How can we find optimal splits for continuous thresholds?

Continuous features



Arrangement of instances over a continuous feature (horizontal axis).

Where to set the thresholds and how many?

Continuous features

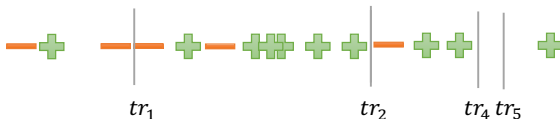


Various possible thresholds to split data over a continuous feature

We can calculate the impurity of splits

How many splits are possible?

Continuous features



Various possible thresholds to split data over a continuous feature

We can calculate the impurity of splits

How many splits are possible?

Two points to consider when limiting the number of thresholds:

Point 1

Only one split is needed in the *space* between two observed values.

Point 2

Only splits between examples from *different classes* can improve the information gain.

Finding the best split for a decision tree

Algorithm *BestSplit-Class*(D, F) – find the best split for a decision tree.

In : data D ; set of features F .

O : feature f to split on.

```
1  $I_{\min} \leftarrow 1$ ;  
2 for each  $f \in F$  do  
3   | split  $D$  into subsets  $D_1, \dots, D_l$  according to the values  $v_j$  of  $f$ ;  
4   | if  $\text{Imp}(\{D_1, \dots, D_l\}) < I_{\min}$  then  
5   |   |  $I_{\min} \leftarrow \text{Imp}(\{D_1, \dots, D_l\})$ ;  
6   |   |  $f_{\text{best}} \leftarrow f$ ;  
7   | end  
8 end  
9 return  $f_{\text{best}}$ 
```

Table of content

1. Tree models
2. Decision trees
 - Growing a decision tree
 - Pruning
 - Sensitivity to skewed class distributions
3. Regression trees

Reducing the size of the tree

The tree will grow to fit the data (i.e., memorize training data, or zero training error) and can become very large → risks of *overfitting* when we have noisy and limited training data.

Options to prevent overfitting:

- *Limit the number of iterations* of the algorithm leading to a tree with a bounded number of nodes.
- Another common solution is to *prune* the tree after it is built by **removing weak** branches, hoping to reduce it to a much smaller tree, but still with a similar empirical error.

Pruning

Reduced error pruning:

- Starting at the leaves, each node is replaced with the majority class.
- If the prediction accuracy is not affected then the change is kept.
- Keep a validation set, see pruned tree performance on the validation set (generalisation). Of course, the pruning will not improve accuracy on the training set.
- Simple

Train pruning



Validate pruning

Reduced error pruning demo

`https://colab.research.google.com/drive/18gKL-VUIipRiC7pOyowOaJqthO-\_uuph`

Table of content

1. Tree models

2. Decision trees

- Growing a decision tree
- Pruning
- Sensitivity to skewed class distributions

3. Regression trees

Imbalances

Sources of imbalance:

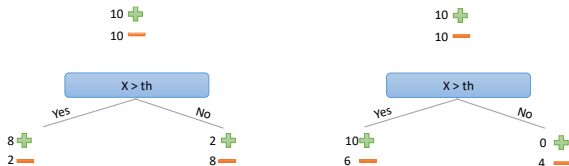
- Asymmetric class distribution
- Asymmetric mis-classification cost

Solutions

Changing the training data:

- Add samples (e.g., oversampling the minority class)
 - Adds data for training and increases the training time
 - It is possible that it does not make any difference

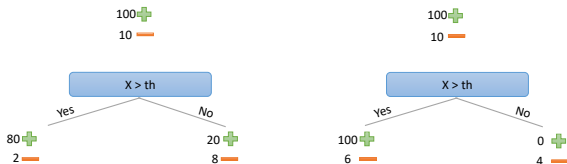
Changing data



Two different splits over continuous feature.

Imagine that the errors on the positive class are 10 times more expensive than the negative class ...

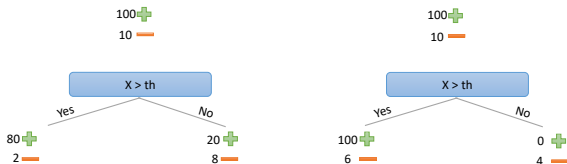
Changing data



Two different splits over continuous feature.

Let's proportionally increase the number of positive examples ...

Changing data



Two different splits over continuous feature.

Let's proportionally increase the number of positive examples ...

→ What would be the effect on the impurity measures?

Relative impurity

Let's imagine that you increase the number of positive examples 10 times. What happens to the relative gains? →

Gini

- Impurity of the parent: $2 \frac{n^{\oplus}}{n} \frac{n^{\ominus}}{n}$
- Weighted impurity of the first child: $\frac{n_1}{n} 2 \frac{n_1^{\oplus}}{n_1} \frac{n_1^{\ominus}}{n_1}$
- Relative impurity: $\frac{n_1^{\oplus} n_1^{\ominus} / n_1}{n^{\oplus} n^{\ominus} / n}$

Relative impurity

Let's imagine that you increase the number of positive examples 10 times. What happens to the relative gains? →

Gini

- Impurity of the parent: $2 \frac{n^{\oplus}}{n} \frac{n^{\ominus}}{n}$
- Weighted impurity of the first child: $\frac{n_1}{n} 2 \frac{n_1^{\oplus}}{n_1} \frac{n_1^{\ominus}}{n_1}$
- Relative impurity: $\frac{n_1^{\oplus} n_1^{\ominus} / n_1}{n^{\oplus} n^{\ominus} / n}$

$\sqrt{\text{Gini}}$

Relative impurity

Let's imagine that you increase the number of positive examples 10 times. What happens to the relative gains? →

Gini

- Impurity of the parent: $2 \frac{n^{\oplus}}{n} \frac{n^{\ominus}}{n}$
- Weighted impurity of the first child: $\frac{n_1}{n} 2 \frac{n_1^{\oplus}}{n_1} \frac{n_1^{\ominus}}{n_1}$
- Relative impurity: $\frac{n_1^{\oplus} n_1^{\ominus} / n_1}{n^{\oplus} n^{\ominus} / n}$

$\sqrt{\text{Gini}}$

- Impurity of the parent: $\sqrt{\frac{n^{\oplus}}{n} \frac{n^{\ominus}}{n}};$
- Weighted impurity of the first child: $\frac{n_1}{n} \sqrt{\frac{n_1^{\oplus}}{n_1} \frac{n_1^{\ominus}}{n_1}};$
- Relative impurity: $\sqrt{\frac{n_1^{\oplus} n_1^{\ominus}}{n^{\oplus} n^{\ominus}}}.$

Relative impurity

- Gini: $\frac{n_1^{\oplus} n_1^{\ominus} / n_1}{n^{\oplus} n^{\ominus} / n}$
- $\sqrt{\text{Gini}}$: $\sqrt{\frac{n_1^{\oplus} n_1^{\ominus}}{n^{\oplus} n^{\ominus}}}$.

$\sqrt{\text{Gini}}$

$\sqrt{\text{Gini}}$ is designed to minimise relative impurity, and thus is **insensitive to changes** in class distribution, whereas Gini (and similarly entropy) emphasises children covering more examples.

Sensitivity to fluctuations in class distributions

Important

Entropy and Gini index are sensitive to fluctuations in the class distribution, $\sqrt{\text{Gini}}$ isn't.

Regression trees

Regression tree

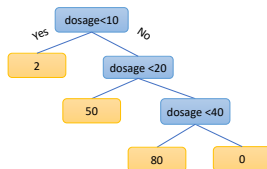
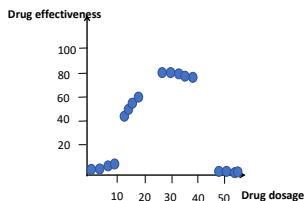
Can we extend decision trees to other tasks (e.g., regression)

Regression tree

Let's start with regression. We have a target variable that is numeric.

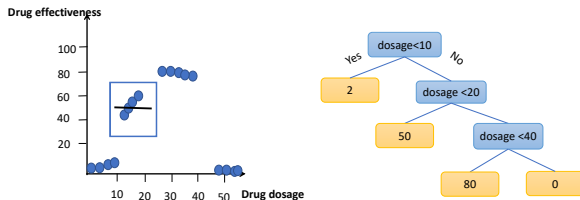
Regression tree

Having one predictor



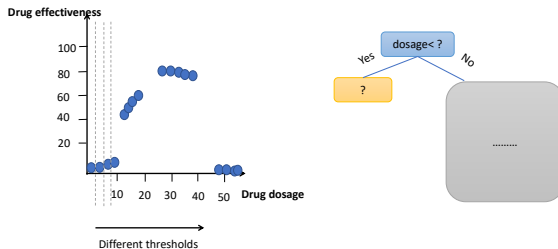
Regression tree

The mean value of instances in a split is used as prediction



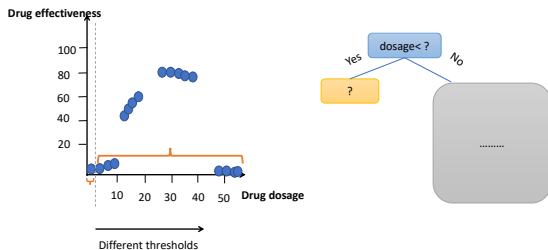
Regression tree

What should be the threshold?



Regression tree

What should be the threshold?



Before we had impurity, what is an equivalent for regression trees?

Regression trees

Regression trees

In regression problems we can replace the *Imp* with the variance of a set Y of target values as the average squared distance from the mean (or the residual).

$$\text{Var}(Y) = \frac{1}{|Y|} \sum_{y \in Y} (y - \bar{y})^2$$

Weighted variance

If a **split** partitions the set of target values Y into mutually exclusive sets $\{Y_1, \dots, Y_l\}$, the weighted average variance is

$$\text{Var}(\{Y_1, \dots, Y_l\}) = \sum_{j=1}^l \frac{|Y_j|}{|Y|} \text{Var}(Y_j)$$

Tree learning as variance reduction

An important point to realise

- The variance of a Boolean (i.e., Bernoulli) variable with success probability p is $p(1 - p)$, which is half the Gini index. So we could interpret the goal of tree learning as minimising the *class variance* (or standard deviation, in case of $\sqrt{\text{Gini}}$) in the leaves.

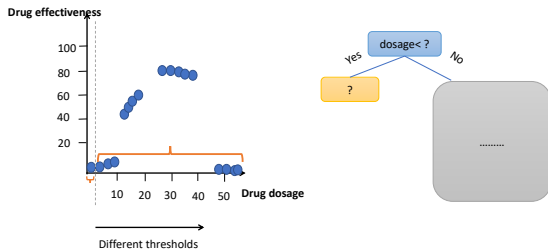
Regression trees

Goal

Our goal is to find a split that minimises the weighted average of the variance of the child nodes.

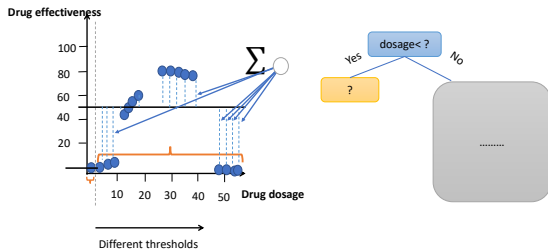
Regression tree

What should be the split?



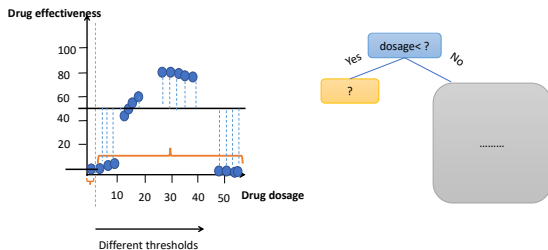
Regression tree

What should be the threshold of the first feature?



Regression tree

What should be the threshold?



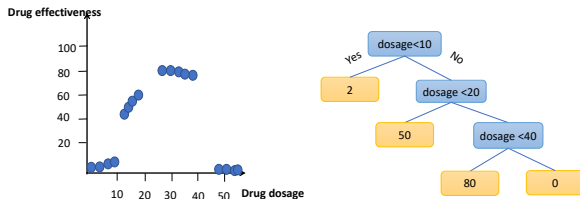
You repeat this for all the other thresholds and see which one gives you a lower variance.

Regression tree

What happens if we have more than one predictor (e.g., dosage, sex, age)?

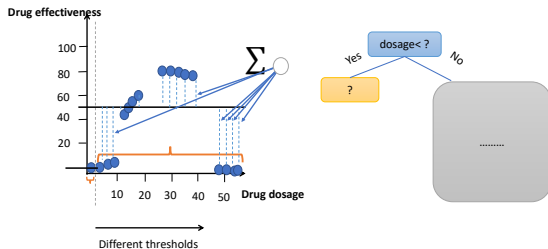
Regression tree

Having more than one predictor (e.g., dosage, sex, age)



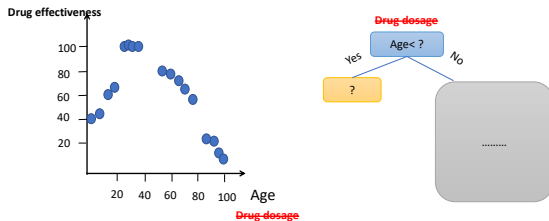
Regression tree

What should be the threshold for the first feature?



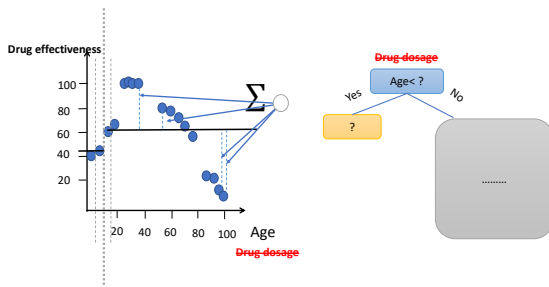
Regression tree

What should be the threshold for the second feature?



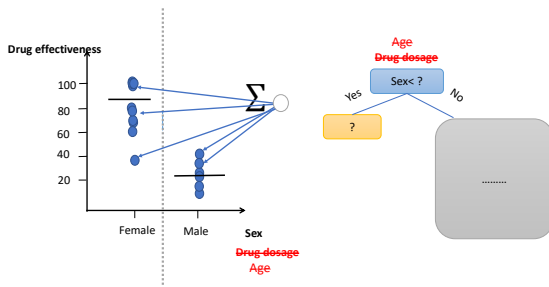
Regression tree

What should be the threshold for the second feature?



Regression tree

What should be the threshold for the third feature?



Regression tree

What should be the threshold?

- First select the best split per feature
- Next compare the features and select the one with minimum variance
- Repeat for each leaf

Lessons learned

- Tree models (decision trees, random forest, gradient boosting)
- Decision trees are widely adopted and interpretable
- Tree learning by finding the best split
 - **Classification trees:** Minority class, Entropy, Gini index
 - **Regression trees:** Variance
- Overfitting in regression trees → pruning